Day 6. Gradient descent for multiple linear regression

Gradient descent for multiple linear regression

	Previous notation	Vector notation
Parameters	$w_1, \cdots, w_n$ b	vector of length n $\vec{w} = [w_1 \ \cdots \ w_n]$ b still a number
Model	$f_{\vec{w},b}(\vec{x}) = w_1x_1 + \cdots + w_nx_n + b$	$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$ dot product
Cost function	$J(w_1, \cdots, w_n, b)$	$J(\vec{w}, b)$

Gradient descent

repeat {
$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(w_1, \cdots, w_n, b)$
$b = b - \alpha \frac{\partial}{\partial b} J(w_1, \cdots, w_n, b)$
}
	repeat {
	$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$
	$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$

Gradient descent

One feature

repeat {

$$w = w - \alpha \underbrace{\frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)x^{(i)}}_{\frac{\partial}{\partial w} J(w,b)}$$

$$b = b - \alpha \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)$$

simultaneously update w, b

}

n features ($n \geq 2$)

repeat {

$j=1$

$$w_1 = w_1 - \alpha \underbrace{\frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)x_1^{(i)}}_{\frac{\partial}{\partial w_1} J(\vec{w},b)}$$

$\vdots$

$j=n$

$$w_n = w_n - \alpha \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)x_n^{(i)}$$

$$b = b - \alpha \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)$$

simultaneously update w_j (for $j = 1, \cdots, n$) and b

}

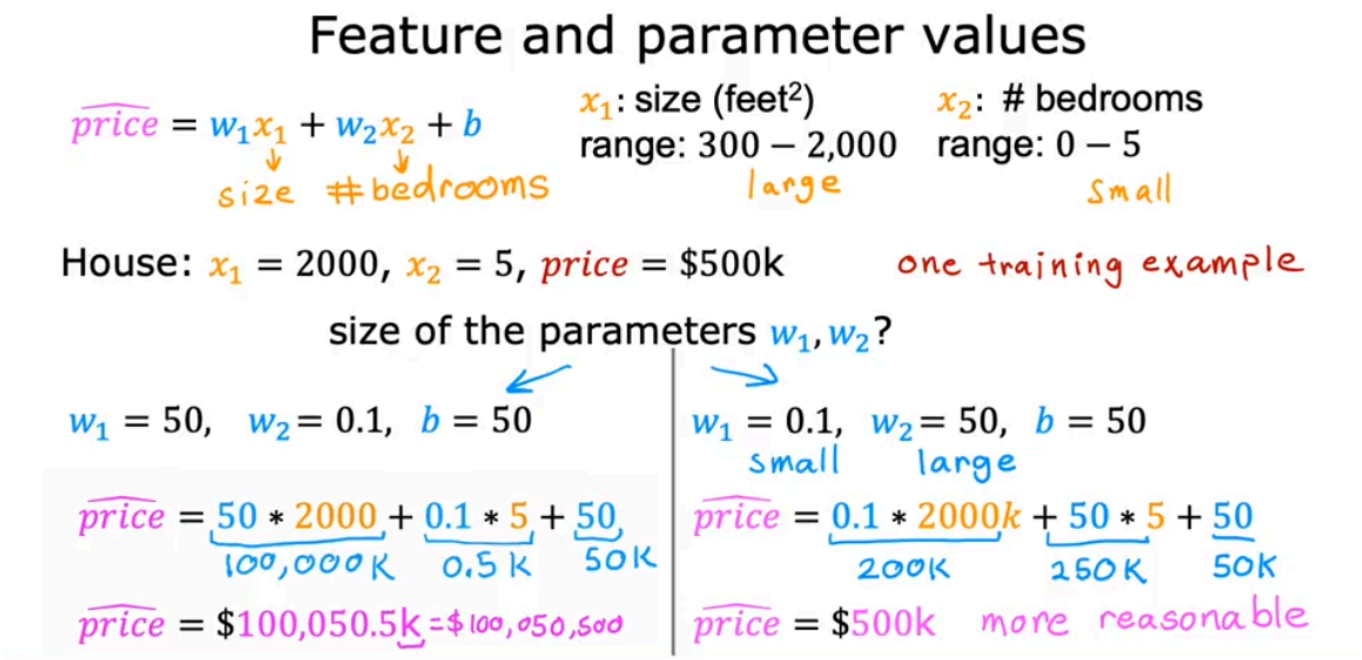
Normal equation

an alternative to gradient descent

- only for linear regression
- solve for w, b without iterations
  Disadvantages

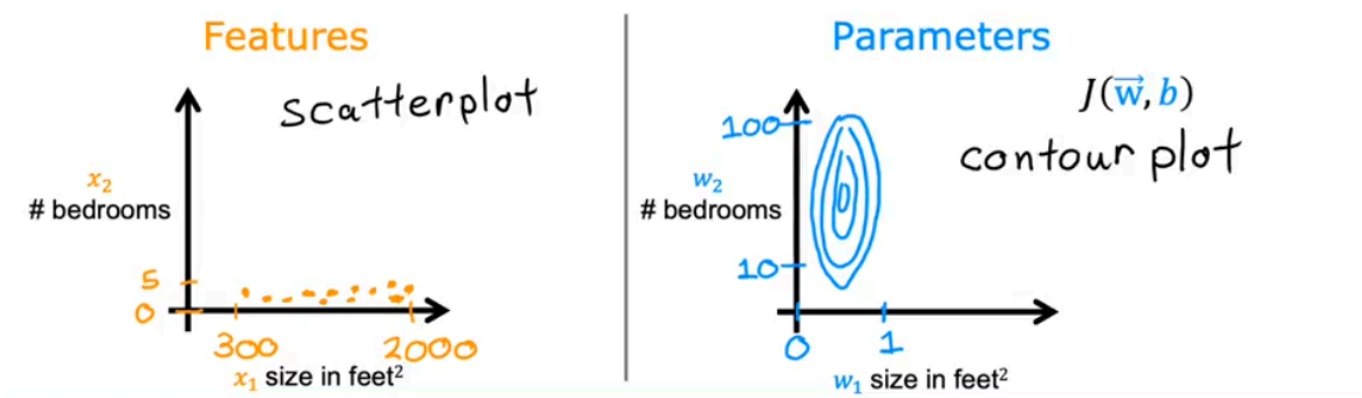
- doesn't generalize to other learning algorithms
- slow when number of features is large
- normal equation mehod may be used in machine learning libraries that implement linear regression
- gradient descent ifs the recommended method for finding parameters w, b.

Feature scaling

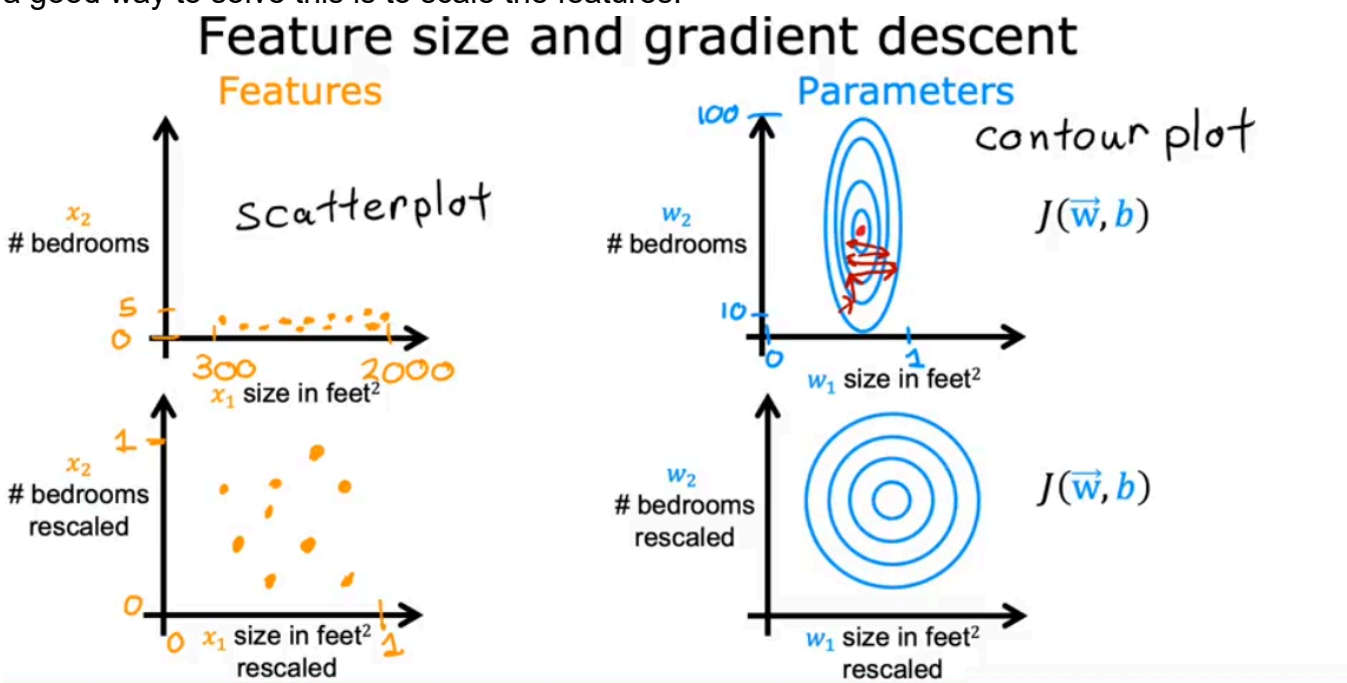


Feature size and parameter size

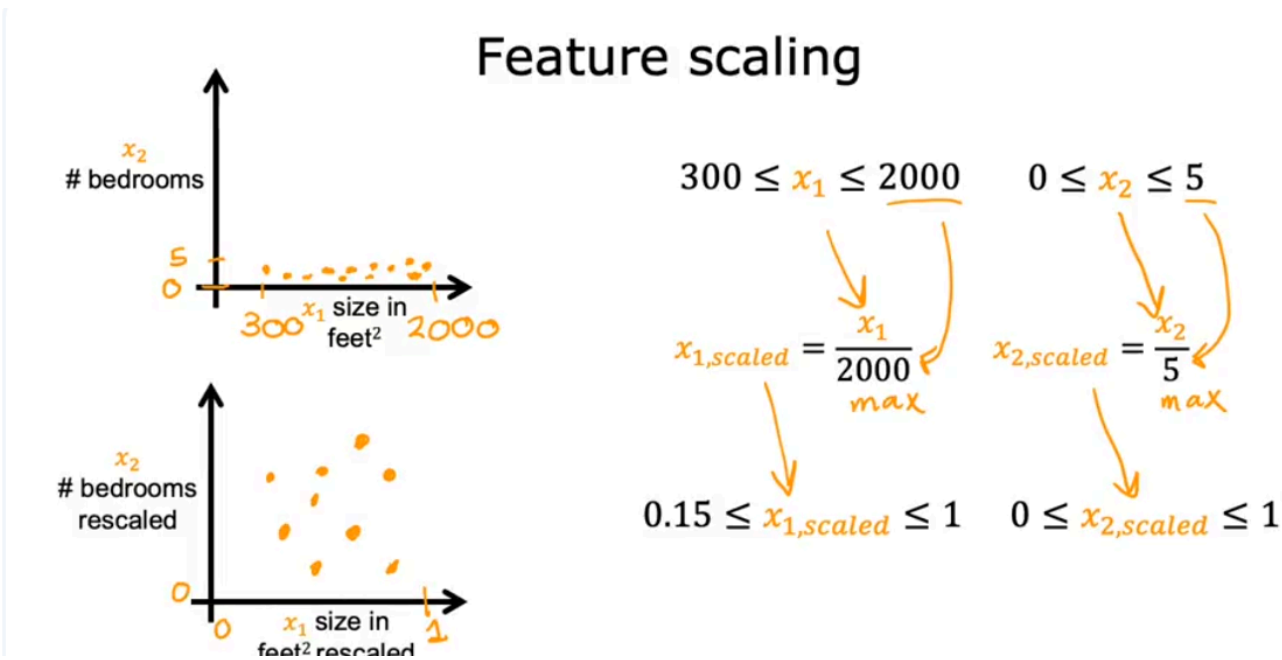
	size of feature x_j	size of parameter w_j
size in feet²	⟷	⟷
#bedrooms	⟷	⟷



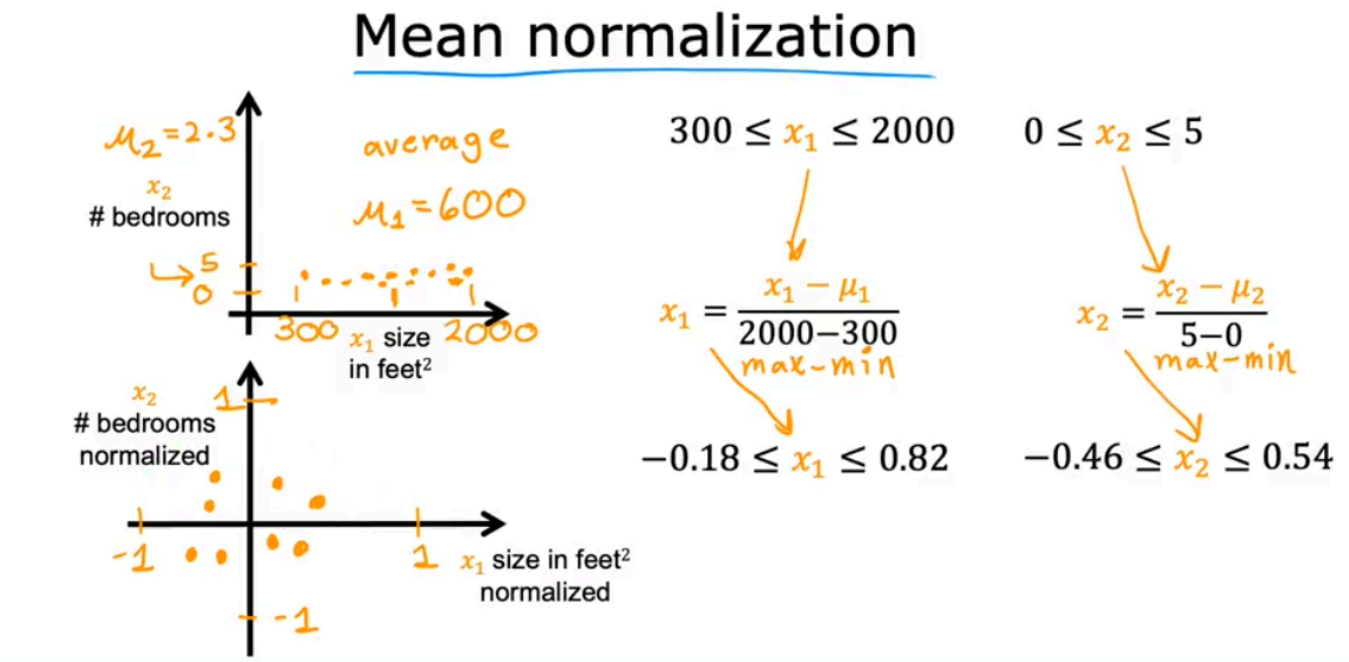
a good way to solve this is to scale the features.



Implement feature scaling

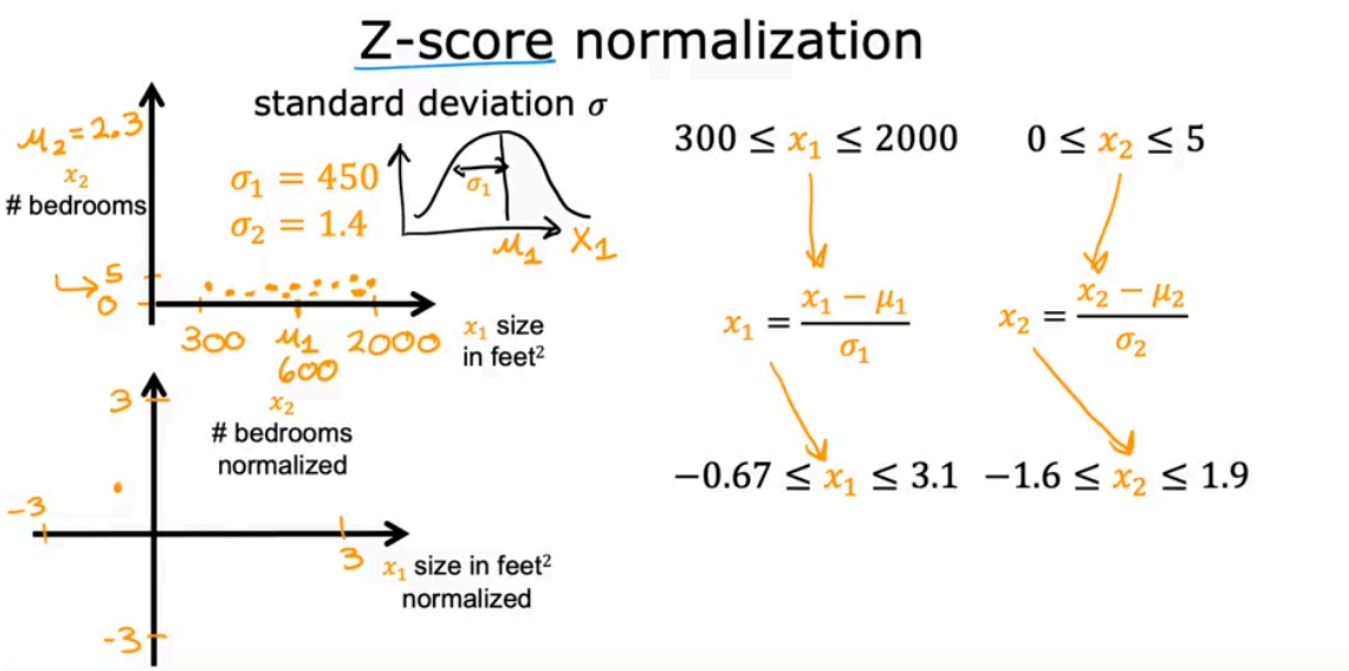


Mean normalization



Z-score normalization

> standard deviation σ?



About scalling

Feature scaling

aim for about $-1 \le x_j \le 1$ for each feature x_j

$-3 \le x_j \le 3$

$-0.3 \le x_j \le 0.3$

} acceptable ranges

$0 \le x_1 \le 3$ okay, no rescaling

$-2 \le x_2 \le 0.5$ okay, no rescaling

$-100 \le x_3 \le 100$ too large → rescale

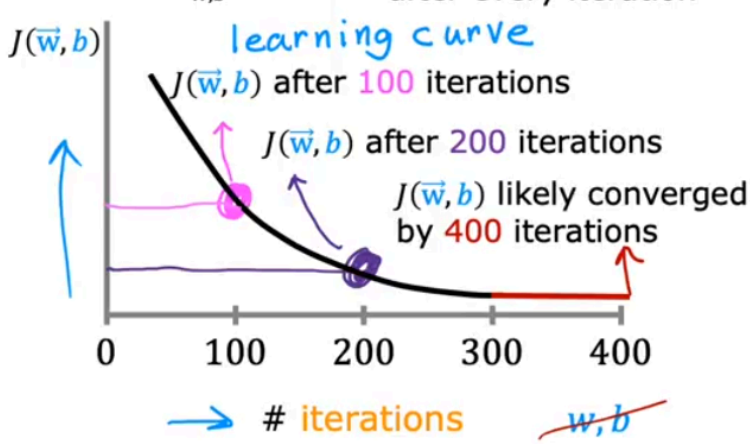
$-0.001 \le x_4 \le 0.001$ too small → rescale

Checking gradient descent for convergence

Make sure gradient descent is working correctly

objective: $\min_{\vec{w},b} J(\vec{w},b)$

$J(\vec{w},b)$ should decrease after every iteration



iterations needed varies 30 1,000 100,000

Automatic convergence test

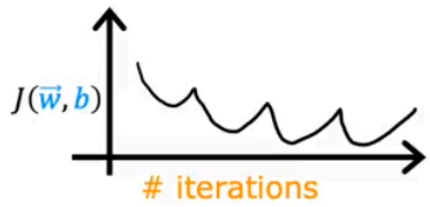
Let ε "epsilon" be 10^{-3}.
0.001

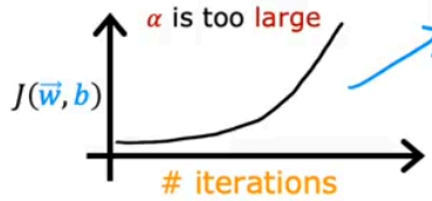
If $J(\vec{w},b)$ decreases by $\le \varepsilon$ in one iteration, declare convergence.

(found parameters $\vec{w},b$ to get close to global minimum)

Choosing the learning rate

Identify problem with gradient descent





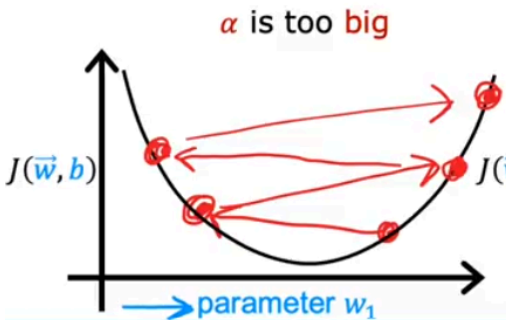
or learning rate is too large

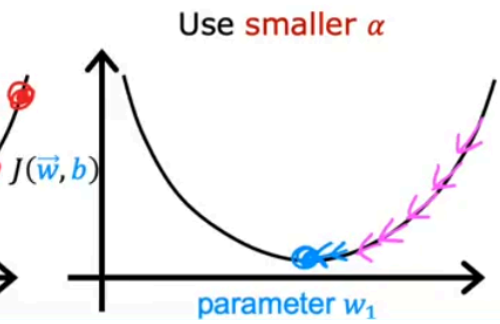
$w_1 = w_1 + \alpha d_1$

use a minus sign

$w_1 = w_1 - \alpha d_1$

Adjust learning rate

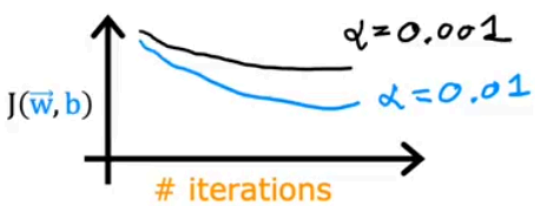
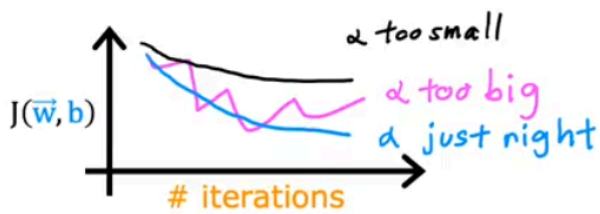




With a small enough α, $J(\vec{w}, b)$ should decrease on every iteration

If α is too small, gradient descent takes a lot more iterations to converge

Values of α to try:

... 0.001 0.003 0.01 0.03 0.1 0.3 1 ...

3X ≈3X 3X ≈3X 3X ≈3X